

Curs 7 - partea 2

Exemple de utilizare a claselor interne în Java

Introducere

În Java avem mai multe tipuri de **clase interne** (nested classes):

- **Clasă internă non-statică (inner class)**: definită în interiorul altei clase, are acces la membrii instanță ai clasei exterioare și este legată logic de o instanță a acesteia.
- **Clasă internă statică (static nested class)**: definită în interiorul altei clase, dar marcată **static**; nu depinde de o instanță a clasei exterioare.
- **Clasă locală**: definită în interiorul unei metode; există și este vizibilă doar în cadrul acelei metode.

1 Problema 1: Modelarea unei mașini cu motor și specificații

1. Să se definească o clasă **Masina** cu:

- câmpuri private: `String model`, `int kilometraj`;
- un constructor care primește ca parametru modelul mașinii;
- o metodă publică `afiseazaRevizie()` ce va folosi o **clasă locală** pentru a afișa informații despre o revizie;
- metoda `main` pentru testare.

2. În interiorul clasei **Masina** să se definească o **clasă internă non-statică** **Motor** care:

- are un câmp privat `boolean pornit`;
- metode publice:
 - `void porneste()` – pornește motorul;
 - `void opreste()` – oprește motorul;
 - `void ruleaza(int km)` – dacă motorul este pornit, crește `kilometraj` cu valoarea `km` și afișează distanța parcursă; altfel, afișează un mesaj de eroare.
- are acces la câmpurile instanței **Masina**.

3. În interiorul clasei **Masina** să se definească o **clasă internă statică** **MotorFactory** care:

- conține metode statice:
 - `MotorSpecifications createStandardSpec();`
 - `MotorSpecifications createSportSpec();`
- aceste metode întorc obiecte de tipul `MotorSpecifications`.

4. În interiorul clasei **MotorFactory** să se definească o **clasă internă statică** **MotorSpecifications** cu:

- câmpuri: `int` `caiPutere`, `String` `combustibil`;
- un constructor cu parametri pentru inițializarea câmpurilor;
- metoda publică `void` `afisare()` pentru afișarea specificațiilor.

5. În metoda `afiseazaRevizie()` din `Masina` să se definească o **clasă locală** `Revizie` cu:

- un câmp `String` `mesaj`;
- un constructor pentru inițializare;
- metoda `void` `tipareste()` pentru afișarea unui mesaj de revizie care să includă modelul mașinii.

6. În metoda `main`:

- se creează o instanță de `Masina`;
- se creează un obiect `Motor` și se testează metodele `porneste`, `ruleaza`, `opreste`;
- se apelează `afiseazaRevizie()`;
- se creează și se afișează specificațiile standard de motor folosind `MotorFactory`.

Soluție

```

1  public class Masina {
2      private String model;
3      private int kilometraj;
4
5      public Masina(String model) {
6          this.model = model;
7          this.kilometraj = 0;
8      }
9      // Clasa interna non-static
10     public class Motor {
11         private boolean pornit = false;
12         public void porneste() {
13             if (!pornit) {
14                 pornit = true;
15                 System.out.println("Motorul a pornit.");
16             }
17         }
18
19         public void opreste() {
20             if (pornit) {
21                 pornit = false;
22                 System.out.println("Motorul a fost oprit.");
23             }
24         }
25
26         public void ruleaza(int km) {
27             if (pornit) {
28                 kilometraj += km;
29                 System.out.println("Masina a parcurs " + km + " km.");
30             } else {
31                 System.out.println("Nu poti rula cu motorul oprit!");
32             }
33         }
34     } // end of class Motor
35

```

```

36 // Clasa statica interna pentru clasa Masina
37 public static class MotorFactory {
38     public static MotorSpecifications createStandardSpec() {
39         return new MotorSpecifications(75, "benzina");
40     }
41     public static MotorSpecifications createSportSpec() {
42         return new MotorSpecifications(150, "benzina premium");
43     }
44 //clasa statica interna pentru clasa MotorFactory
45 public static class MotorSpecifications {
46     int caiPutere;
47     String combustibil;
48     public MotorSpecifications(int cp, String combustibil) {
49         this.caiPutere = cp;
50         this.combustibil = combustibil;
51     }
52     public void afisare() {
53         System.out.println("Specificatii motor:");
54         System.out.println("- " + caiPutere + " CP");
55         System.out.println("- combustibil: " + combustibil);
56     }
57 } //end clasa MotorSpecifications
58 } //end clasa MotorFactory
59
60 // Metoda care foloseste o CLASA LOCALA
61 public void afiseazaRevizie() {
62     // Clasa LOCALA : exista doar in aceasta metoda
63     class Revizie {
64         private String mesaj;
65         Revizie(String mesaj) {
66             this.mesaj = mesaj;
67         }
68         public void tipareste() {
69             System.out.println("Revizie pentru " + model + ": " + mesaj)
70             ;
71         }
72     } //end clasa Revizie
73     Revizie r = new Revizie("schimb ulei + verificare frane.");
74     r.tipareste();
75 }
76 public static void main(String[] args) {
77     Masina m = new Masina("Dacia Logan");
78     Masina.Motor motor = m.new Motor();
79     motor.pornește();
80     motor.ruleaza(40);
81     motor.opreste();
82
83     m.afiseazaRevizie();
84
85     // Clasa statica
86     Masina.MotorFactory.MotorSpecifications spec = Masina.
87         MotorFactory.createStandardSpec();
88     spec.afisare();
89 } //end main
90 } //end clasa Masina

```

Explicații

- **Clasa internă non-statică Motor:**
 - este definită în interiorul clasei `Masina`;
 - are acces direct la câmpurile instanței `Masina` (`kilometraj`, `model`);
 - este creată prin `Masina.Motor motor = m.new Motor()`; ceea ce arată dependența de instanța `m`.
- **Clasa internă statică MotorFactory:**
 - nu depinde de o instanță de `Masina`;
 - conține doar metode statice care creează obiecte `MotorSpecifications`;
 - poate fi utilizată direct prin `Masina.MotorFactory.createStandardSpec()`.
- **Clasa internă statică MotorSpecifications:**
 - este definită în interiorul clasei statice `MotorFactory`;
 - grupează informațiile despre un anumit tip de motor (putere, combustibil);
 - metoda `afisare()` prezintă clar aceste informații.
- **Clasa locală Revizie:**
 - este definită în interiorul metodei `afiseazaRevizie()`;
 - poate accesa câmpurile instanței exterioare (`model`);
 - este folosită doar pentru a afișa un mesaj de revizie, deci nu are sens în afara metodei.

2 Problema 2: Modelarea unei case cu camere și senzori

1. Să se definească o clasă `Casa` cu:
 - câmp privat: `String adresa`;
 - un constructor care primește adresa;
 - o metodă publică `afiseazaRaportCasa()` ce va folosi o **clasă locală** pentru a afișa un raport despre casă;
 - metoda `main` pentru testare.
2. În interiorul clasei `Casa` să se definească o **clasă internă non-statică** `Camera` cu:
 - câmp privat: `String numeCamera`;
 - un constructor care primește numele camerei;
 - metoda publică `void descriere()` pentru a afișa numele camerei.
3. În interiorul clasei `Camera` să se definească o **clasă internă non-statică** `Senzor` cu:
 - câmp privat: `String tip`;
 - un constructor care primește tipul senzorului;
 - metoda publică `void citeste()` care afișează un mesaj de citire a valorii senzorului și menționează numele camerei.
4. În interiorul clasei `Casa` să se definească o **clasă internă statică** `PlanCasa` cu:
 - metoda statică `void afiseazaPlanStandard()` ce afișează un plan standard al casei;

- în interiorul acesteia, o clasă internă statică `CameraDefault` cu:
 - câmpuri: `String nume`, `double suprafata`;
 - un constructor cu parametri;
 - metoda `void info()` pentru afișarea informațiilor camerei standard.
5. În metoda `afiseazaRaportCasa()` din `Casa` să se definească o **clasă locală** `Raport` cu:
- câmp `String titlu`;
 - constructor pentru inițializare;
 - metoda `void tipareste()` ce afișează titlul și adresa casei.
6. În metoda `main`:
- se creează o instanță de `Casa`;
 - se creează o cameră (de exemplu, "Living") și un senzor (de exemplu, "Temperatură") în acea cameră;
 - se apelează metodele de descriere a camerei și de citire a senzorului;
 - se apelează `afiseazaRaportCasa()`;
 - se apelează `PlanCasa.afiseazaPlanStandard()` și se creează o `CameraDefault` pentru a afișa informațiile unei camere standard.

Soluție

```

1  public class Casa {
2
3      private String adresa;
4
5      public Casa(String adresa) {
6          this.adresa = adresa;
7      }
8
9      // =====
10     // Clasa interna non-static
11     // =====
12     public class Camera {
13         private String numeCamera;
14
15         public Camera(String numeCamera) {
16             this.numeCamera = numeCamera;
17         }
18
19         // Clasa interna in Camera
20         public class Senzor {
21             private String tip;
22
23             public Senzor(String tip) {
24                 this.tip = tip;
25             }
26
27             public void citeste() {
28                 System.out.println(
29                     "Senzorul " + tip +
30                     " din camera " + numeCamera +
31                     " citeste o valoare..."

```

```

32         );
33     }
34 }
35
36     public void descriere() {
37         System.out.println("Camera: " + numeCamera);
38     }
39 }
40
41 // =====
42 // Clasa statica interna
43 // =====
44 public static class PlanCasa {
45
46     public static void afiseazaPlanStandard() {
47         System.out.println("Plan standard: 3 camere, baie,
48             bucatarie.");
49     }
50
51     public static class CameraDefault {
52         private String nume;
53         private double suprafata;
54
55         public CameraDefault(String nume, double suprafata) {
56             this.nume = nume;
57             this.suprafata = suprafata;
58         }
59
60         public void info() {
61             System.out.println("Camera default: " + nume);
62             System.out.println("Suprafata: " + suprafata + " mp"
63                 );
64         }
65     }
66 }
67
68 // =====
69 // Metoda cu o CLASA LOCALA
70 // =====
71 public void afiseazaRaportCasa() {
72
73     // Clasa LOCALA
74     class Raport {
75         private String titlu;
76
77         Raport(String titlu) {
78             this.titlu = titlu;
79         }
80
81         public void tipareste() {
82             System.out.println("=== " + titlu + " ===");
83             System.out.println("Adresa: " + adresa);
84         }
85     }
86
87     Raport r = new Raport("Raport Casa");
88     r.tipareste();
89 }

```

```

88
89
90     public static void main(String[] args) {
91
92         Casa c = new Casa("Strada Florilor 12");
93
94         Casa.Camera living = c.new Camera("Living");
95         Casa.Camera.Senzor senzorTemp = living.new Senzor("
96             Temperatura");
97
98         living.descriere();
99         senzorTemp.citeste();
100
101         c.afiseazaRaportCasa();
102
103         Casa.PlanCasa.afiseazaPlanStandard();
104         Casa.PlanCasa.CameraDefault cd =
105             new Casa.PlanCasa.CameraDefault("Dormitor", 14.2);
106         cd.info();
107     }

```

Explicații

- **Clasa internă non-statică Camera:**
 - este legată de o instanță de Casa;
 - este creată cu `Casa.Camera living = c.new Camera("Living");`;
 - reprezintă o componentă logică a casei.
- **Clasa internă non-statică Senzor:**
 - este definită în interiorul clasei Camera;
 - are acces la `numeCamera`;
 - este creată cu `living.new Senzor("Temperatură");`.
- **Clasa internă statică PlanCasa:**
 - nu depinde de o instanță concretă de Casa;
 - oferă informații generale (plan standard) despre tipuri de case.
- **Clasa internă statică CameraDefault:**
 - descrie caracteristicile unei camere standard;
 - este creată cu `new Casa.PlanCasa.CameraDefault(...)` și afișează informațiile prin metoda `info()`.
- **Clasa locală Raport:**
 - este definită în interiorul metodei `afiseazaRaportCasa()`;
 - are acces la câmpul `adresa` al obiectului Casa;
 - se ocupă exclusiv de formatarea și afișarea raportului.